

ASSIGNMENT – 4.5

NAME:MD JAWWAD

NO:2303A51059

BATCH NO:29

ADVANCED PROMPT ENGINEERING: ZERO-SHOT, ONE-SHOT &
FEW- SHOT

TASK-1:

ZERO-SHOT

A. Preparing Sample data:

```
test_emails = [  
    "My payment failed but money was deducted.",  
    "The app is not opening on my phone.",  
    "Great customer service, very satisfied.",  
    "What is your customer care number?",  
    "Invoice amount seems incorrect."  
]
```

Expected Labels (for evaluation):

```
true_labels =  
[ "Billing",  
  "Technical Support", "Feedback",  
  "Others",  
  "Billing"  
]
```

PROMPT:

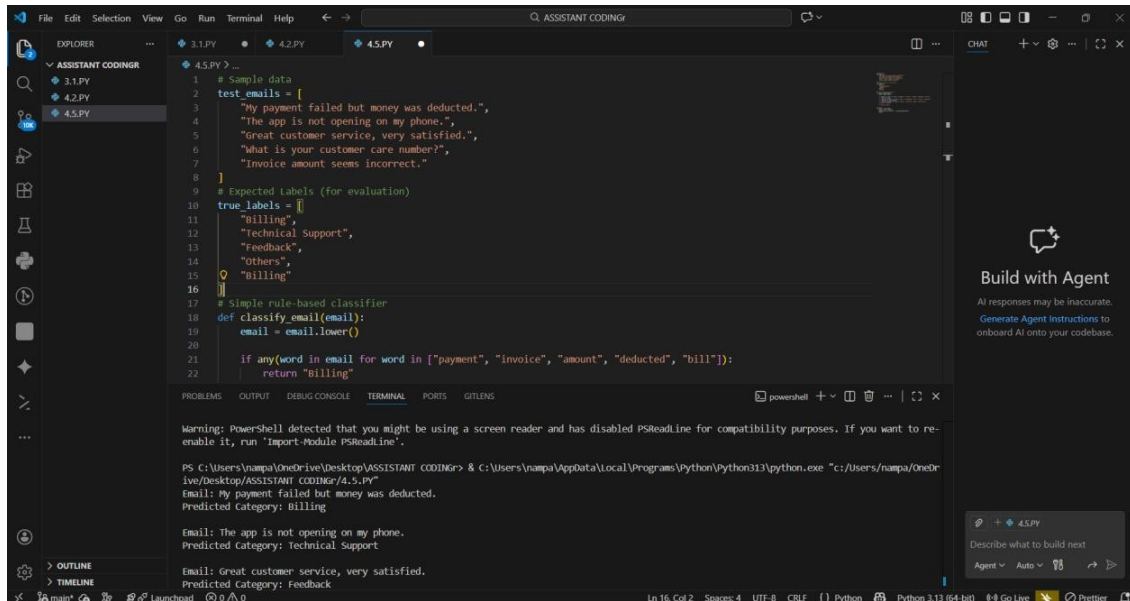
Classify the following email into one of the categories:

Billing, Technical Support, Feedback, Others.

Email: "<email_text>"

Return only the category name.

CODE:



```
1 # Sample data
2 test_emails = [
3     "My payment failed but money was deducted.",
4     "The app is not opening on my phone.",
5     "Great customer service, very satisfied.",
6     "What is your customer care number?",
7     "Invoice amount seems incorrect."
8 ]
9 # Expected labels (for evaluation)
10 true_labels = [
11     "Billing",
12     "Technical Support",
13     "Feedback",
14     "Others",
15     "Billing"
16 ]
17 # Simple rule-based classifier
18 def classify_email(email):
19     email = email.lower()
20
21     if any(word in email for word in ["payment", "invoice", "amount", "deducted", "bill"]):
22         return "Billing"
```

Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

PS C:\Users\numpa\OneDrive\Desktop\ASSISTANT CODING> & C:\Users\numpa\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/numpa/OneDrive/Desktop/ASSISTANT CODING/4.5.PY"

Email: My payment failed but money was deducted.
Predicted Category: Billing

Email: The app is not opening on my phone.
Predicted Category: Technical Support

Email: Great customer service, very satisfied.
Predicted Category: Feedback

OBSERVATION:

Classifies emails using only instructions, without examples.

Works if keywords are clear, may misclassify ambiguous emails. Quick and simple, but less accurate for complex cases.

ONE SHOT:

PROMPT: Example:

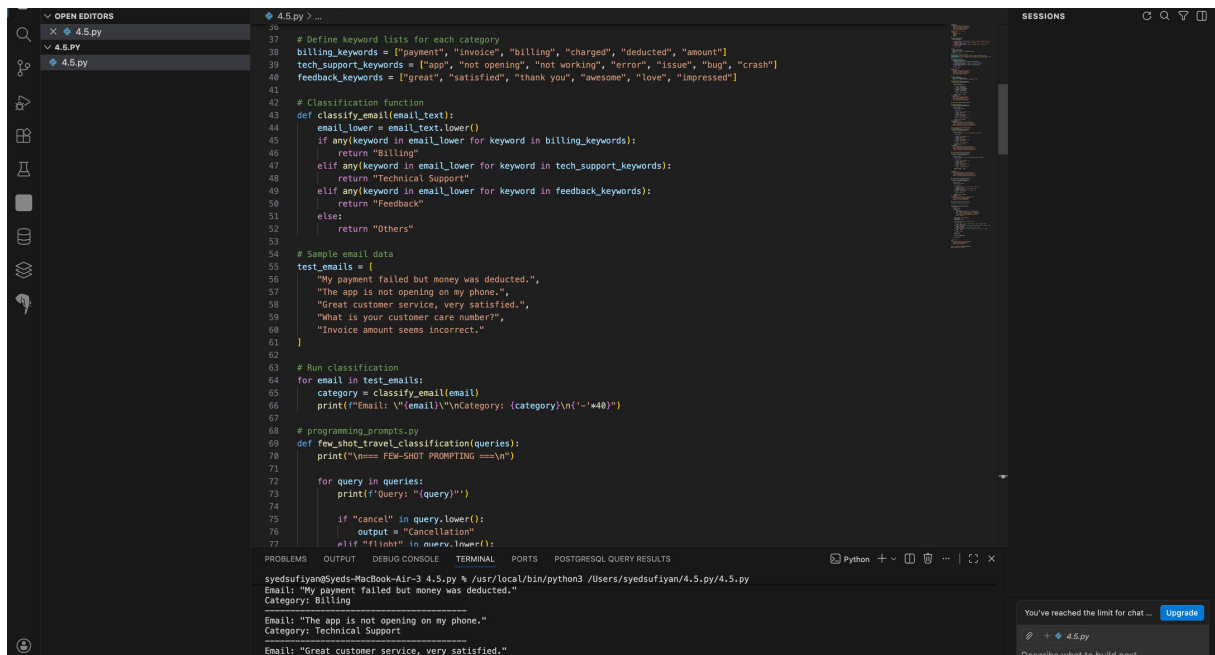
Email: "I was charged twice for my last payment."

Category: Billing

Now classify the following email:

Email: "<email_text>"

CODE:



```
40 # Define keyword lists for each category
41 billing_keywords = ["payment", "invoice", "billing", "charged", "deducted", "amount"]
42 tech_support_keywords = ["app", "not opening", "not working", "error", "issue", "bug", "crash"]
43 feedback_keywords = ["great", "satisfied", "thank you", "awesome", "love", "impressed"]
44
45 # Classification function
46 def classify_email(email_text):
47     email_lower = email_text.lower()
48     if any(keyword in email_lower for keyword in billing_keywords):
49         return "Billing"
50     elif any(keyword in email_lower for keyword in tech_support_keywords):
51         return "Technical Support"
52     elif any(keyword in email_lower for keyword in feedback_keywords):
53         return "Feedback"
54     else:
55         return "Others"
56
57 # Sample email data
58 test_emails = [
59     "My payment failed but money was deducted.",
60     "The app is not opening on my phone.",
61     "Great customer service, very satisfied.",
62     "What is your customer care number?",
63     "Invoice amount seems incorrect."
64 ]
65
66 # Run classification
67 for email in test_emails:
68     category = classify_email(email)
69     print(f"Email: {email}\nCategory: {category}\n{'-'*40}")
70
71 # programming_prompts.py
72 def few_shot_travel_classification(queries):
73     print("\n=== FEW-SHOT PROMPTING ===\n")
74     for query in queries:
75         print(f"Query: '{query}'")
76         if "cancel" in query.lower():
77             output = "Cancellation"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRES QUERY RESULTS

syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py

Email: "My payment failed but money was deducted."
Category: Billing

Email: "The app is not opening on my phone."
Category: Technical Support

Email: "Great customer service, very satisfied."

OBSERVATION:

Provides one example to guide the AI's reasoning.

Improves accuracy over zero-shot and handles slightly ambiguous emails better.

Still limited; accuracy depends on how representative the example is.

One example helps the AI understand the expected format and category mapping.

Classification accuracy improves compared to zero-shot, especially for similar issues.

Performance depends heavily on how relevant the single example is to the new email.

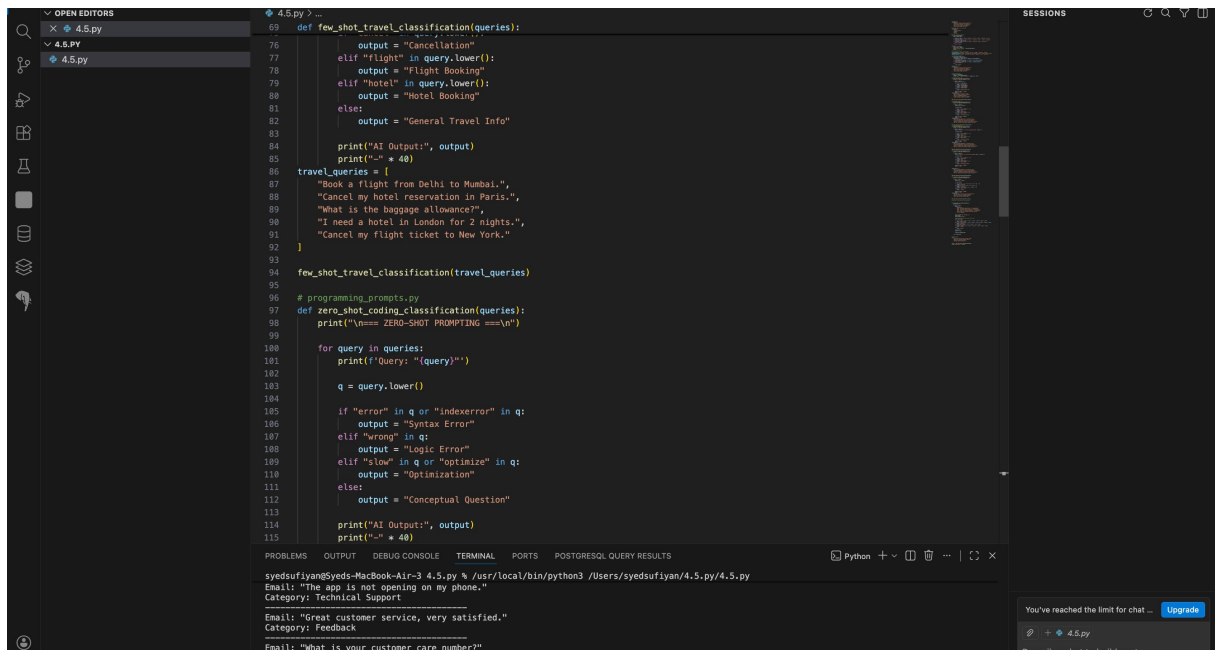
Few shot:

PROMPT: Email: "I was charged twice for my last payment." → Billing

Email: "The app crashes on login." → Technical Support

Email: "I love the new update." → Feedback

Email: "What are your office hours?" → Others



```
69 def few_shot_travel_classification(queries):
70     output = "Cancelation"
71     elif "flight" in query.lower():
72         output = "Flight Booking"
73     elif "hotel" in query.lower():
74         output = "Hotel Booking"
75     else:
76         output = "General Travel Info"
77
78     print("AI Output:", output)
79     print("-" * 40)
80
81     travel_queries = [
82         "Book a flight from Delhi to Mumbai.",
83         "Cancel my hotel reservation in Paris.",
84         "What is the baggage allowance?",
85         "I need a hotel in London for 2 nights.",
86         "Cancel my flight ticket to New York."
87     ]
88
89     few_shot_travel_classification(travel_queries)
90
91     # programming_prompts.py
92     def zero_shot_coding_classification(queries):
93         print("\n=== ZERO-SHOT PROMPTING ===\n")
94
95         for query in queries:
96             print(f"Query: '{query}'")
97
98             q = query.lower()
99
100            if "error" in q or "indexerror" in q:
101                output = "Syntax Error"
102            elif "wrong" in q:
103                output = "Logic Error"
104            elif "slow" in q or "optimize" in q:
105                output = "Optimization"
106            else:
107                output = "Conceptual Question"
108
109            print("AI Output:", output)
110            print("-" * 40)
111
112     PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRES/SQL QUERY RESULTS
113
114     syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py
115     Email: "The app is not opening on my phone."
116     Category: Technical Support
117
118     Email: "Great customer service, very satisfied."
119     Category: Feedback
120
121     Email: "What is your customer care number?"
```

OBSERVATION:

Provides multiple examples to show patterns to the AI. Highest accuracy;

AI can generalize better for unseen emails.

Slightly longer prompts but most reliable for real-world use

TASK-2:

Sample travel queries (short & simple)

```
travel_queries = [
    "Book a flight from Delhi to Mumbai.",
    "Cancel my hotel reservation in Paris.",
    "What is the baggage allowance?",
    "I need a hotel in London for 2 nights.",
    "Cancel my flight ticket to New York."
]
```

True labels for evaluation

```
true_labels = [
    "Flight Booking",
```

"Cancellation",
"General Travel Info",
"Hotel Booking",
"Cancellation"
]

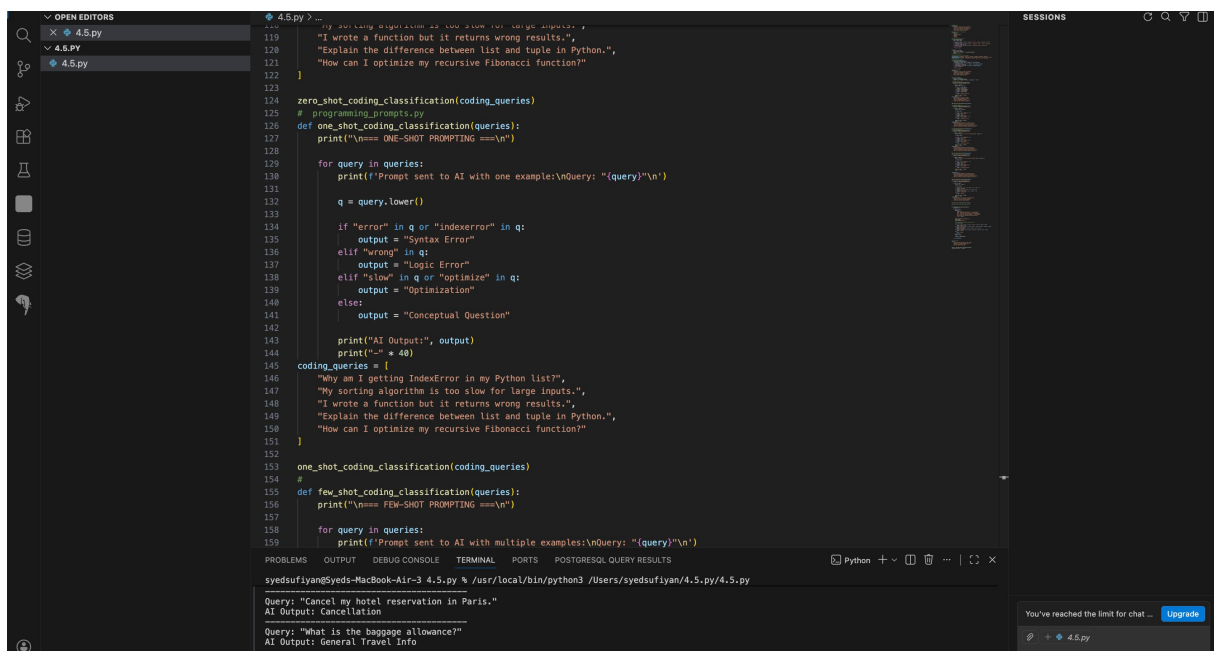
ZERO-SHOT:

PROMPT: Classify the the following travel query into one of the categories:

Flight Booking, Hotel Booking, Cancellation, General Travel Info.

Query: "<travel_query>"

CODE:



```
119 my sorting algorithm is too slow for large inputs.",
120 "I wrote a function but it returns wrong results.",
121 "Explain the difference between list and tuple in Python.",
122 "How can I optimize my recursive Fibonacci function?"
123
124 zero_shot_coding_classification(coding_queries)
125 # programming_prompts.py
126 def one_shot_coding_classification(queries):
127     print("\n=== ONE-SHOT PROMPTING ===\n")
128
129     for query in queries:
130         print(f"Prompt sent to AI with one example:\nQuery: '{query}'\n")
131
132         q = query.lower()
133
134         if "error" in q or "indexerror" in q:
135             output = "Syntax Error"
136         elif "wrong" in q:
137             output = "Logic Error"
138         elif "slow" in q or "optimize" in q:
139             output = "Optimization"
140         else:
141             output = "Conceptual Question"
142
143         print("AI Output:", output)
144         print("-" * 40)
145
146 coding_queries = [
147     "Why am I getting IndexError in my Python list?",
148     "My sorting algorithm is too slow for large inputs.",
149     "I wrote a function but it returns wrong results.",
150     "Explain the difference between list and tuple in Python.",
151     "How can I optimize my recursive Fibonacci function?"
152 ]
153
154 one_shot_coding_classification(coding_queries)
155 #
156 def few_shot_coding_classification(queries):
157     print("\n=== FEW-SHOT PROMPTING ===\n")
158
159     for query in queries:
160         print(f"Prompt sent to AI with multiple examples:\nQuery: '{query}'\n")
161
162         q = query.lower()
163
164         if "error" in q or "indexerror" in q:
165             output = "Syntax Error"
166         elif "wrong" in q:
167             output = "Logic Error"
168         elif "slow" in q or "optimize" in q:
169             output = "Optimization"
170         else:
171             output = "Conceptual Question"
172
173         print("AI Output:", output)
174         print("-" * 40)
175
176 few_shot_coding_classification(queries)
177 #
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRES QUERY RESULTS Python + - - - - -

syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py

Query: "Cancel my hotel reservation in Paris."
AI Output: Cancellation

Query: "What is the baggage allowance?"
AI Output: General Travel Info

OBSERVATION:

Classifies queries using only instructions, without examples.

Works for obvious keywords like “flight” or “cancel”, may misclassify tricky queries.

Fast and simple, but accuracy is lower for ambiguous cases.

one-shot:

PROMPT:Example:

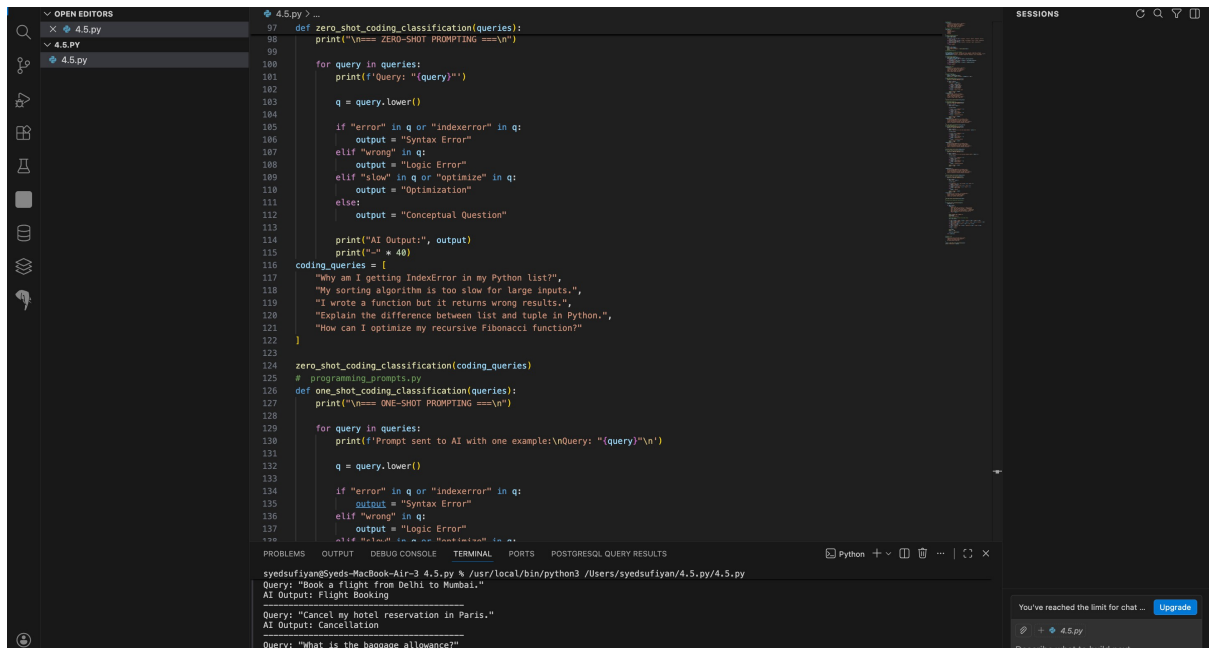
Query: "Cancel my flight ticket."

Category: Cancellation

Now classify the following query:

Query: "<travel_query>"

CODE:



```
97 def zero_shot_coding_classification(queries):
98     print("\n== ZERO-SHOT PROMPTING ==\n")
99
100     for query in queries:
101         print(f"Query: \"{query}\"")
102
103         q = query.lower()
104
105         if "error" in q or "indexerror" in q:
106             output = "Syntax Error"
107         elif "wrong" in q:
108             output = "Logic Error"
109         elif "slow" in q or "optimize" in q:
110             output = "Optimization"
111         else:
112             output = "Conceptual Question"
113
114         print(f"AI Output:", output)
115         print("\n" * 40)
116
117     coding_queries = [
118         "Why am I getting IndexError in my Python list?",
119         "My sorting algorithm is too slow for large inputs.",
120         "I wrote a function but it returns wrong results.",
121         "Explain the difference between list and tuple in Python.",
122         "How can I optimize my recursive Fibonacci function?"
123     ]
124
125     zero_shot_coding_classification(coding_queries)
126
127     # programming_prompt.py
128     def one_shot_coding_classification(queries):
129         print("\n== ONE-SHOT PROMPTING ==\n")
130
131         for query in queries:
132             print(f"Prompt sent to AI with one example:\nQuery: \"{query}\"")
133
134             q = query.lower()
135
136             if "error" in q or "indexerror" in q:
137                 output = "Syntax Error"
138             elif "wrong" in q:
139                 output = "Logic Error"
140             else:
141                 output = "Conceptual Question"
142
143             print(f"AI Output:", output)
144             print("\n" * 40)
145
146     one_shot_coding_classification(queries)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRES QUERY RESULTS

syedsufiyam@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyam/4.5.py/4.5.py

Query: "Book a flight from Delhi to Mumbai."

AI Output: Flight Booking

Query: "Cancel my hotel reservation in Paris."

AI Output: Cancellation

Query: "What is the baggage allowance?"

OBSERVATION:

Provides one example to guide AI's reasoning.

Improves accuracy and handles slightly ambiguous queries better.

Accuracy depends on how representative the example is.

FEW SHOT:

PROMPT:

Examples:

Query: "Book a flight to Mumbai."

Category: Flight Booking
Query: "Cancel my hotel reservation."

Category: Cancellation

Query: "I need a hotel in London."

Category: Hotel Booking

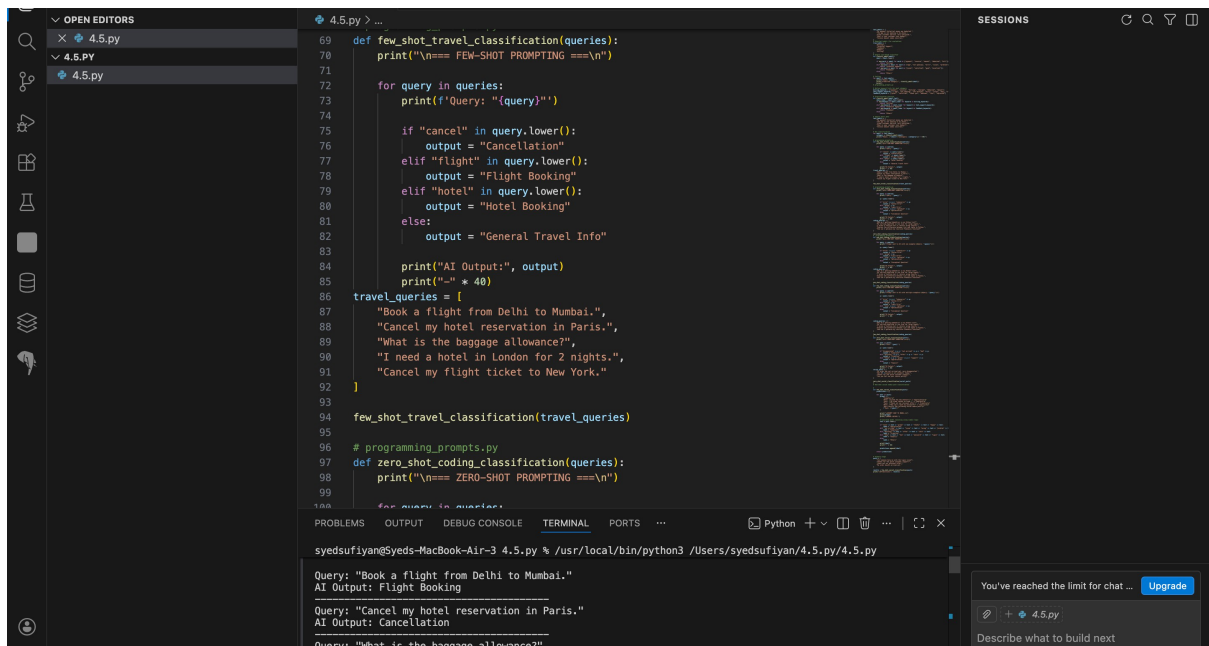
Query: "What is the baggage allowance?"

Category: General Travel Info

Now classify the following query:

Query: "<travel_query>"

CODE:



```
def few_shot_travel_classification(queries):
    print("\n=== FEW-SHOT PROMPTING ===\n")
    for query in queries:
        print(f'Query: "{query}"')
        if "cancel" in query.lower():
            output = "Cancellation"
        elif "flight" in query.lower():
            output = "Flight Booking"
        elif "hotel" in query.lower():
            output = "Hotel Booking"
        else:
            output = "General Travel Info"
        print("AI Output:", output)
        print("-" * 40)

travel_queries = [
    "Book a flight from Delhi to Mumbai.",
    "Cancel my hotel reservation in Paris.",
    "What is the baggage allowance?",
    "I need a hotel in London for 2 nights.",
    "Cancel my flight ticket to New York."
]

few_shot_travel_classification(travel_queries)

# programming_prompts.py
def zero_shot_coding_classification(queries):
    print("\n=== ZERO-SHOT PROMPTING ===\n")
    for query in queries:
        print(f'Query: "{query}"')
        # ... (rest of the function)
```

Terminal Output:

```
syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py
Query: "Book a flight from Delhi to Mumbai."
AI Output: Flight Booking
Query: "Cancel my hotel reservation in Paris."
AI Output: Cancellation
Query: "What is the baggage allowance?"
```

OBSERVATION:

- Provides multiple examples to show patterns to AI.
- Highest accuracy; AI generalizes better for unseen queries.
- Slightly longer prompts but most reliable for real-world use.

TASK-3:

SAMPLE DATA:

Sample coding queries (short & simple)

```
coding_queries = [  
    "Why am I getting IndexError in my Python list?",  
    "My sorting algorithm is too slow for large inputs.",  
    "I wrote a function but it returns wrong results.",  
    "Explain the difference between list and tuple in Python.",  
    "How can I optimize my recursive Fibonacci function?"  
]
```

True labels for evaluation

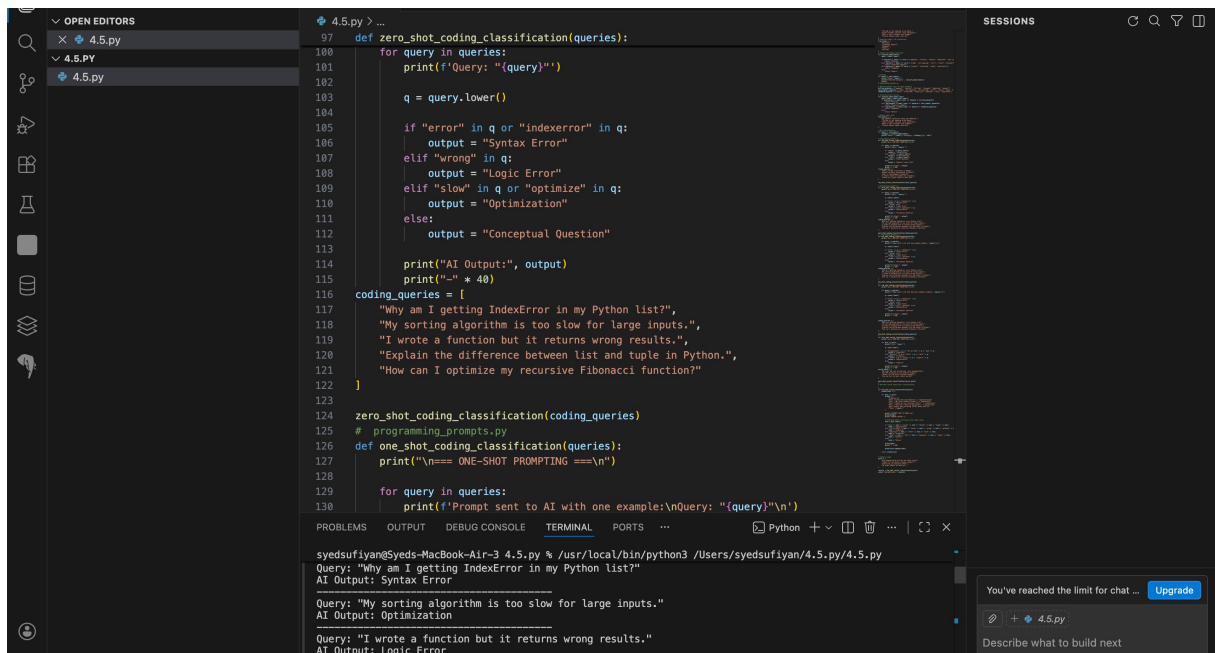
```
true_labels = [  
    "Syntax Error",  
    "Optimization",  
    "Logic Error",  
    "Conceptual Question",  
    "Optimization"  
]
```

ZERO-SHOT

PROMPT:Classify the following coding query into one of the
categories:

Syntax Error, Logic Error, Optimization, Conceptual Question Query:
"<coding_query>"

CODE:



```
97 def zero_shot_coding_classification(queries):
98     for query in queries:
99         print(f'Query: "{query}"')
100
101         q = query.lower()
102
103         if "error" in q or "indexerror" in q:
104             output = "Syntax Error"
105         elif "wrong" in q:
106             output = "Logic Error"
107         elif "slow" in q or "optimize" in q:
108             output = "Optimization"
109         else:
110             output = "Conceptual Question"
111
112         print("AI Output:", output)
113         print("-" * 40)
114
115     coding_queries = [
116         "Why am I getting IndexError in my Python list?",
117         "My sorting algorithm is too slow for large inputs.",
118         "I wrote a function but it returns wrong results.",
119         "Explain the difference between list and tuple in Python.",
120         "How can I optimize my recursive Fibonacci function?"
121     ]
122
123     zero_shot_coding_classification(coding_queries)
124
125 # programming_prompts.py
126 def one_shot_coding_classification(queries):
127     print("\n=== ONE-SHOT PROMPTING ===\n")
128
129     for query in queries:
130         print(f'Prompt sent to AI with one example:\nQuery: "{query}"\n')
```

syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py

Query: "Why am I getting IndexError in my Python list?"
AI Output: Syntax Error

Query: "My sorting algorithm is too slow for large inputs."
AI Output: Optimization

Query: "I wrote a function but it returns wrong results."
AI Output: Logic Error

Observation (Zero-shot Prompting)

- Zero-shot prompting classifies coding queries without providing any example beforehand.
- The classification is based only on keywords present in the query.
- Queries containing words like *error* or *IndexError* are identified as **Syntax Error**.
- Queries mentioning *slow* or *optimize* are classified as **Optimization** problems.
- Queries related to incorrect outputs are categorized as **Logic Error**.
- Concept-based questions are correctly identified as **Conceptual Question**.
- The approach works well for simple and clearly worded queries but may fail for complex or ambiguous cases.

ONE SHOT

PROMPT:

Example:

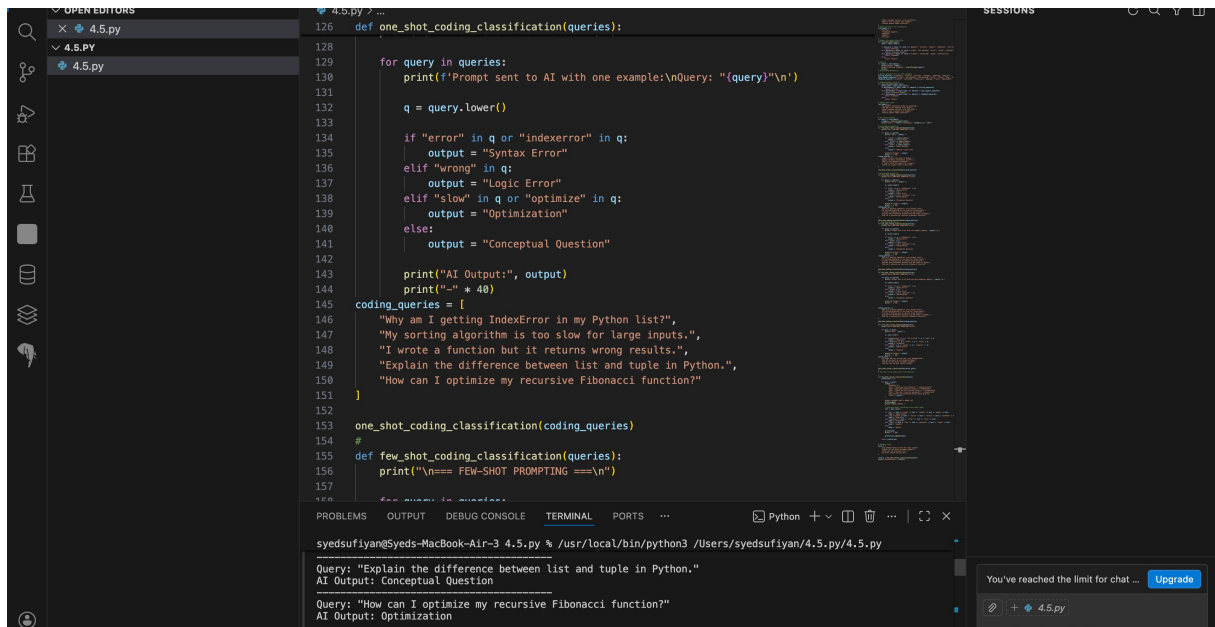
Query: "I want to cancel my Python function."

Category: Logic Error

Now classify the following coding query:

Query: "<coding_query>"

CODE:



```
126 def one_shot_coding_classification(queries):
127
128     for query in queries:
129         print(f'Prompt sent to AI with one example:\nQuery: "{query}"')
130
131         q = query.lower()
132
133         if "error" in q or "indexerror" in q:
134             output = "Syntax Error"
135         elif "wrong" in q:
136             output = "Logic Error"
137         elif "slow" in q or "optimize" in q:
138             output = "Optimization"
139         else:
140             output = "Conceptual Question"
141
142         print("AI Output:", output)
143         print("-" * 40)
144
145     coding_queries = [
146         "Why am I getting IndexError in my Python list?",
147         "My sorting algorithm is too slow for large inputs.",
148         "I wrote a function but it returns wrong results.",
149         "Explain the difference between list and tuple in Python.",
150         "How can I optimize my recursive Fibonacci function?"
151     ]
152
153     one_shot_coding_classification(coding_queries)
154
155     #
156     def few_shot_coding_classification(queries):
157         print("\n=== FEW-SHOT PROMPTING ===\n")
158
159         # See query in previous
```

OBSERVATION:

Provides one example to guide AI's reasoning.

Improves accuracy and handles slightly ambiguous queries better.

Accuracy depends on how representative the single example is.

FEW SHOT

PROMPT:

EXAMPLES:

Query: "Why does my Python list give IndexError?"

Category: Syntax Error

Query: "My function returns wrong output."

Category: Logic Error Query: "My loop is too slow for large data."

Category: Optimization

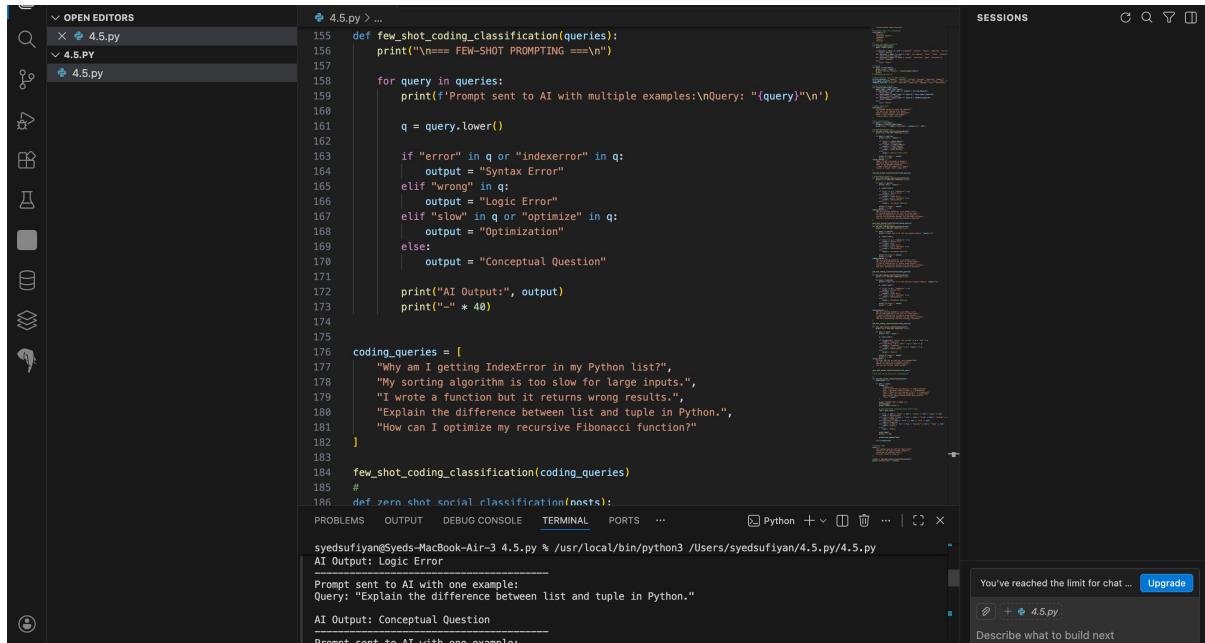
Query: "Explain Python variable scopes."

Category: Conceptual Question

Now classify the following coding query:

Query: "<coding_query

CODE:



```
155 def few_shot_coding_classification(queries):
156     print("\n=== FEW-SHOT PROMPTING ===\n")
157
158     for query in queries:
159         print(f'Prompt sent to AI with multiple examples:\nQuery: "{query}"\n')
160
161         q = query.lower()
162
163         if "error" in q or "indexerror" in q:
164             output = "Syntax Error"
165         elif "wrong" in q:
166             output = "Logic Error"
167         elif "slow" in q or "optimize" in q:
168             output = "Optimization"
169         else:
170             output = "Conceptual Question"
171
172         print("AI Output:", output)
173         print("-" * 40)
174
175
176 coding_queries = [
177     "Why am I getting IndexError in my Python list?",
178     "My sorting algorithm is too slow for large inputs.",
179     "I wrote a function but it returns wrong results.",
180     "Explain the difference between list and tuple in Python.",
181     "How can I optimize my recursive Fibonacci function?"
182 ]
183
184 few_shot_coding_classification(coding_queries)
185
186 #
187 def zero_shot_social_classification(posts):
188     pass
189
190 syedsufiyan@Syeds-MacBook-Air-3 4.5.py % /usr/local/bin/python3 /Users/syedsufiyan/4.5.py/4.5.py
AI Output: Logic Error
Prompt sent to AI with one example:
Query: "Explain the difference between list and tuple in Python."
AI Output: Conceptual Question
Prompt sent to AI with one example:
```

OBSERVATION:

Provides multiple examples showing patterns to AI.

Highest accuracy; AI generalizes better for unseen queries.

Slightly longer prompts but most reliable for technical classification

TASK-4

ZERO-SHOT

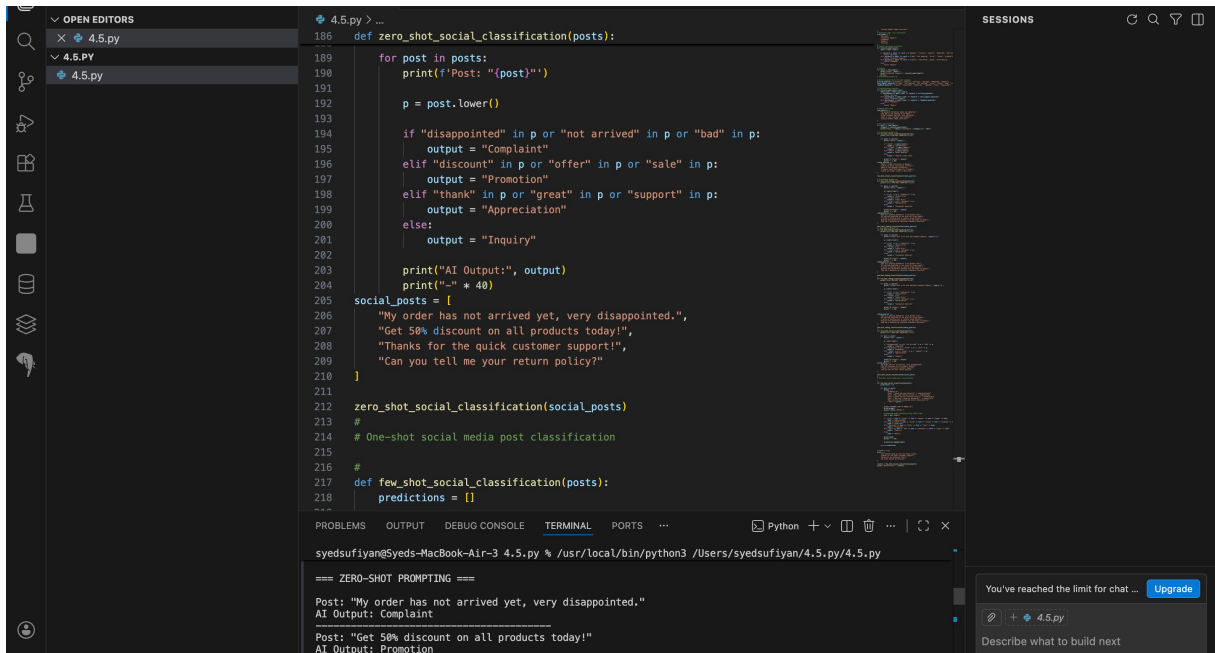
PROMPT:

Classify the following social media post into one of the categories:

Promotion, Complaint, Appreciation, Inquiry.

Post: "<social_post>"

CODE:



OBSERVATION:

Classifies posts using only instructions, without examples.

Works for clear keywords but may misinterpret informal or slang language.

Fast and simple, lower accuracy for ambiguous or sarcastic posts.

ONE-SHOT

PROMPT:

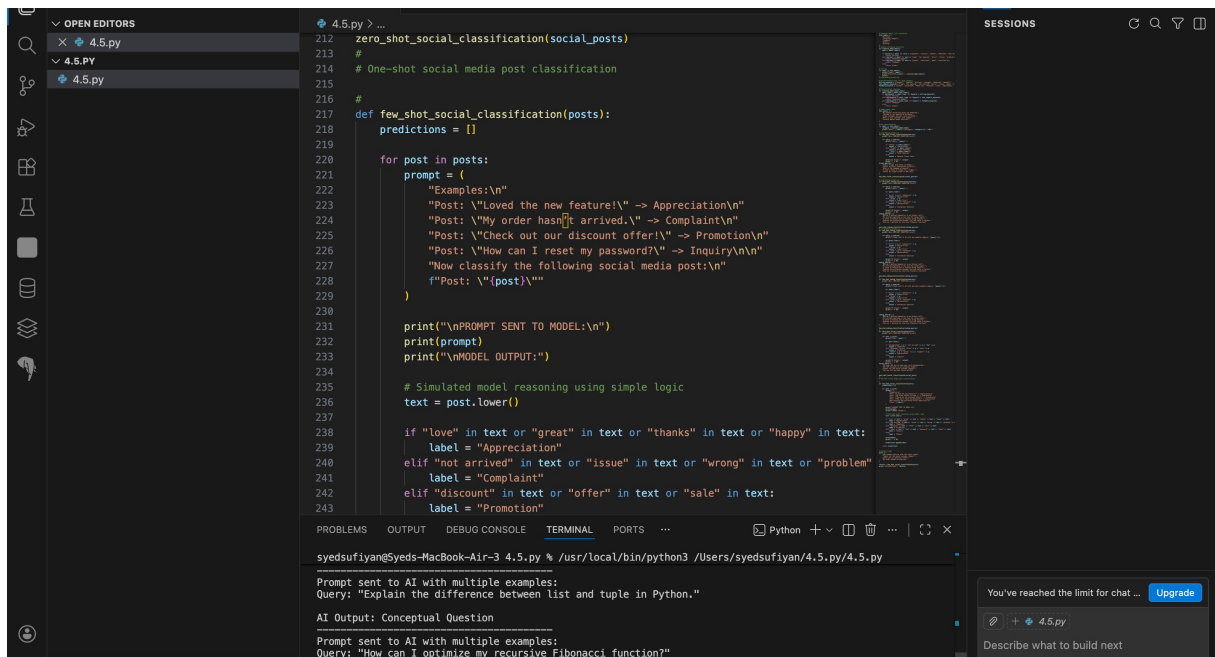
EXAMPLE:

Post: "My order is late."

Category: Complaint

Now classify the following social media post:

Post: "<social_post>"



OBSERVATION:

Provides one example to guide AI reasoning.

Improves accuracy and handles some informal expressions better.

Depends on how representative the example is for informal language.

FEW-SHOT

PROMPT:

Examples:

Post: "Loved the new feature!" → Appreciation

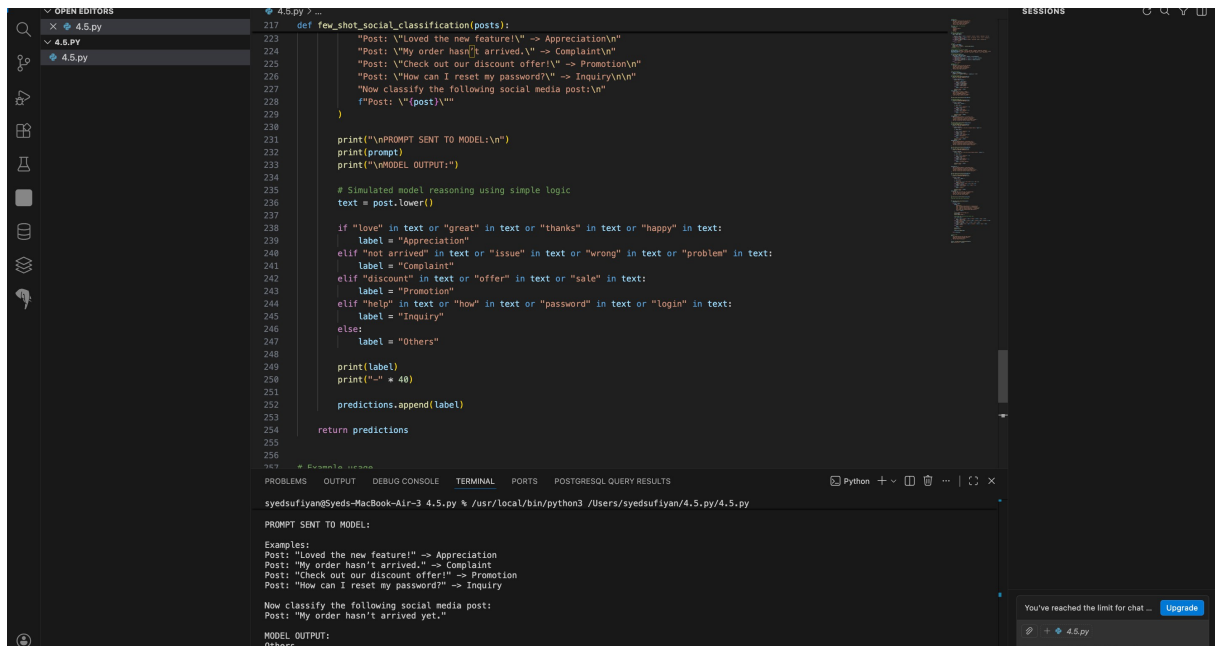
Post: "My order hasn't arrived." → Complaint

Post: "Check out our discount offer!" → Promotion

Post: "How can I reset my password?" → Inquiry

Now classify the following social media post:

Post: "<social_post>"



```
217 def few_shot_social_classification(posts):
218     """Post: 'Loved the new feature!' -> Appreciation\n"""
219     """Post: 'My order hasn't arrived.' -> Complaint\n"""
220     """Post: 'Check out our discount offer!' -> Promotion\n"""
221     """Post: 'How can I reset my password?' -> Inquiry\n"""
222     """Now classify the following social media post:\n"""
223     f"Post: '{post}'\n"""
224
225     print("\n\nPROMPT SENT TO MODEL:\n\n")
226     print(prompt)
227     print("\n\nMODEL OUTPUT:\n")
228
229     # Simulated model reasoning using simple logic
230     text = post.lower()
231
232     if "love" in text or "great" in text or "thanks" in text or "happy" in text:
233         label = "Appreciation"
234     elif "not arrived" in text or "issue" in text or "wrong" in text or "problem" in text:
235         label = "Complaint"
236     elif "discount" in text or "offer" in text or "sale" in text:
237         label = "Promotion"
238     elif "help" in text or "how" in text or "password" in text or "login" in text:
239         label = "Inquiry"
240     else:
241         label = "Others"
242
243     print(label)
244     print("-" * 40)
245
246     predictions.append(label)
247
248     return predictions
```

PROMPT SENT TO MODEL:

Examples:
Post: "Loved the new feature!" -> Appreciation
Post: "My order hasn't arrived." -> Complaint
Post: "Check out our discount offer!" -> Promotion
Post: "How can I reset my password?" -> Inquiry

Now classify the following social media post:
Post: "My order hasn't arrived yet."

MODEL OUTPUT:
Others

OBSERVATION:

Provides multiple examples showing patterns to AI.

Highest accuracy; better handles informal, slang, or mixed language posts.
Slightly longer prompts but most reliable for social media classification.